

Première partie (10 points) -> copie n°1

Exercice 1 – Pour chacun des extraits de code suivants, donner les affichages produits lors de l'appel des lignes numérotées 1 à 11. Préciser si une exception est levée (raison et type de l'exception).

<pre> 1 = list(range(0, 10, 2)) print(1)¹ print(1[-1])² print(1[-4:4])³ print([i for i in 1 if i % 4 == 0])⁴ </pre>	<pre> def surprise(func): def wrapper(*args, **kwargs): print(f"Nb args = {len(args) + len(kwargs)}") res = func(*args) return res return wrapper @surprise def fonction(*args): if len(args) == 1: return str(args[0]) else: return str(args[0]) + fonction(*args[1:]) print(fonction(2, "toto"))¹⁰ print(fonction(1, 2, a=12))¹¹ </pre>
<pre> def carre(n): print(n ** 2) print(carre(5) == 25)⁵ print(carre('a'))⁶ </pre>	
<pre> notes = {'etu1': 12, 'etu2': 14, 'etu2': 16, 'etu3': 20} print(notes.get('etu2', 0))⁷ print(notes.get('etu6', 0))⁸ print(notes['etu6'])⁹ </pre>	

Exercice 2 – Une compagnie de transports souhaite garder un historique des trajets effectués par ses trains en enregistrant les résultats dans un fichier. Ci-contre un exemple de fichier résultat.

```

TER001|A_L_HEURE|0
TGV001|ANNULE
TER001|EN_RETARD|15
TER001|EN_RETARD|90
TER002|A_L_HEURE|0

```

- Écrire une fonction **moyenne()** qui prend en paramètres une liste d'entiers et en retourne la moyenne.
- Écrire une fonction **ajouter_train(numero, statut, **kwargs)** qui prend en paramètres obligatoires un numéro de train et son statut parmi [ANNULE, EN_RETARD, A_L_HEURE]. Si le statut est EN_RETARD le paramètre **retard** (int) devra être renseigné. Si le statut est A_L_HEURE **retard** vaudra 0. Si le statut est ANNULE on ignore le retard. Si le statut renseigné n'est pas correct on lèvera une exception. Si le statut est EN_RETARD mais que le paramètre **retard** n'est pas renseigné on lèvera une exception. Le nom du fichier sera déterminé à partir du paramètre **nom_fichier** s'il est fourni. S'il n'est pas fourni on enregistrera les données dans **trains.txt**. Cette fonction ajoute au fichier une ligne sous la forme **nom_train | statut | retard**.
- Après quelques semaines, le fichier **trains.txt** s'est bien rempli. Écrire une fonction **parser_train()** qui prend en paramètre une ligne représentant un train et la retourne sous forme d'un dictionnaire contenant les clés **numero**, **statut** et **retard**. La clé **retard** ne sera présente que si le train n'est pas annulé.
Ex.: `parser_train("TER001|EN_RETARD|15\n")`
-> `{'nom': 'TER001', 'statut': 'EN_RETARD', 'retard': 15}`
- Écrire une fonction **lire_retards()** qui prend en paramètre un nom de fichier et retourne un dictionnaire dont les clés sont les noms des trains et les valeurs des retards. On ignore les trains annulés.
Ex.: `lire_retards("trains.txt")` # -> `{'TER001': [0, 15, 90], 'TER002': [0]}`
- Écrire une fonction **trop_en_retard()** qui prend en paramètre le dictionnaire construit en 4 et un seuil (en minutes). Pour chaque train, si la moyenne des retards est supérieure à seuil, afficher : *Trop en retard: nom_train (retard_moyen minutes)*

Partie 2 - TrickBand (10 points) -> copie n°2

Le but de cet exercice est de développer quelques-unes des classes servant à implémenter TrickyBand, une application de studio musical multipistes. Dans cette application, les projets musicaux sont composés d'un ensemble de pistes représentées par des objets de la classe **Piste** (voir Figure 1). Chaque piste est constituée d'une suite de régions consécutives positionnées dans le temps. L'horizon de temps est discrétisé de sorte à ce qu'une position particulière sur une piste ou dans un fichier audio, ou bien la durée d'une région soient représentées par un entier de type **int**.

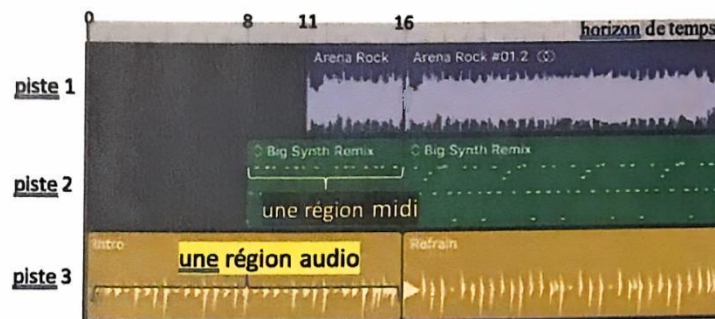


Figure 1 : Illustration empruntée au « Guide d'utilisation de GarageBand, 2023, Apple Inc

Une région est représentée par un objet de la classe **Region** caractérisé par l'attribut **titre** de type **str** permettant de l'identifier dans le visuel de l'application (par exemple « Intro » dans l'image ci-dessus), et un attribut **position** représentant la position de la région sur une piste (par ex 0 pour la région « Intro », ou 11 pour la région « Arena Rock »). La classe **Region** possède un constructeur à 1 seul paramètre permettant d'initialiser le titre ; l'attribut **position** étant initialisé à 0. Si l'argument fourni au constructeur n'est pas de type **str**, cette méthode déclenche une exception.

1. Définir la classe **Region** en respectant les contraintes mentionnées ci-dessus. Vous ferez en sorte de respecter le principe d'encapsulation. Cette classe possèdera des propriétés **titre** et **position** permettant de connaître et de modifier ces attributs. La propriété **position** en modification déclenche une exception si le paramètre fourni n'est pas un entier positif ou nul. L'instruction **print(r)** où **r** est un objet de type **Region** doit provoquer l'affichage de son titre.

Il existe en fait plusieurs types de région : des « régions audio », des « régions MIDI » et des régions « Drummer ». Dans cet exercice, on ne s'intéressera qu'aux régions audios. Une région audio représentée par un objet de la classe **RegionAudio**, correspond à l'utilisation d'une portion d'un fichier audio (représentée par un début et une fin dans le fichier). Le fichier associé est représenté par l'attribut **fichier** de type **Fichier** (qui n'est pas à définir dans cet exercice). La classe **Fichier** propose la méthode **get_duree()** renvoyant un **int** correspondant à sa durée. Elle possède aussi d'autres membres qui ne sont pas abordés dans ce sujet. La classe **RegionAudio** possède deux attributs **debut** et **fin** qui représentent la portion du fichier audio qui est utilisée (on ne définit pas d'accesseurs pour ces attributs). La classe possède un constructeur avec deux paramètres correspondants au titre et au fichier associés à la région. Ce constructeur vérifiera que le 2^e argument proposé est bien de type **Fichier** et déclenchera une exception dans le cas contraire. Les attributs **debut** et **fin** seront initialisés à 0. La méthode **set_portion()** permet de modifier les attributs **debut** et **fin** en fournissant les deux valeurs correspondantes. Cette méthode déclenche une exception, si le début est supérieur ou égal à la fin, ou si la fin précisée est supérieure à la durée du fichier audio. La méthode **get_duree()** de la classe **RegionAudio** renvoie la durée d'une région égale à **fin - debut**. La méthode **get_position_fin()** renvoie la valeur de la position ajoutée à la durée. L'instruction **print(r)** où **r** est un objet de type **RegionAudio** doit provoquer l'affichage de son titre et de sa durée.

2. Définir la classe **RegionAudio** en respectant les contraintes mentionnées ci-dessus. On tiendra compte du fait qu'un objet **RegionAudio** possède toutes les caractéristiques d'un objet **Region**.

La durée globale d'un projet, appelée « horizon du projet », est considérée comme étant un moment où toutes les régions de toutes les pistes sont terminées. Cet horizon peut être agrandi (mais jamais diminué) manuellement, avec la méthode **set_horizon()** expliquée ci-dessous, ou automatiquement lorsqu'on modifie la position d'une région.

De manière à connaître à tout instant l'horizon du projet, il est décidé de faire en sorte que l'ensemble des objets **Region** mettent à jour un attribut partagé **horizon** de type **int**. Initialement (quand il n'y a encore aucune région sur aucune piste), l'horizon est égal à 0. À chaque fois que la position d'une région change, l'objet **Region** concerné doit vérifier si sa position de fin (égale à sa position + sa durée) dépasse l'horizon courant, et mettre à jour éventuellement l'horizon si c'est le cas. Les méthodes de classe **get_horizon()** et **set_horizon()** de la classe **Region** doivent permettre de connaître l'horizon ou d'agrandir manuellement l'horizon en indiquant une valeur donnée (une tentative de diminution n'ayant aucun effet). Ces méthodes peuvent être utilisées de n'importe où dans le programme sans disposer nécessairement d'une instance de **Region**.

3. Modifier la classe **Region** de manière à mettre en œuvre cette nouvelle fonctionnalité.

La classe **Piste** possède un constructeur sans argument. Les régions d'une piste sont stockées dans l'attribut **regions** de type **list**. Initialement, un objet **Piste** ne contient aucune région. L'instruction **pt.add_region(t, file, p, d, f)** permet de créer une nouvelle région **r** de titre **t**, associée au fichier **file** utilisé sur la portion de **d** à **f**, et positionnée à la date **p** dans la piste **pt**. Cette méthode insère la région **r** dans la liste **regions**. On souhaite que la liste **regions** soit triée par position. On prendra donc soin d'insérer la nouvelle région au bon endroit pour éviter les chevauchements : si la fin d'une région se retrouve après le début de la région suivante, il faut décaler cette seconde. Notez qu'il est possible qu'un décalage entraîne un autre. L'instruction **print(pt)** provoque l'affichage de l'ensemble des régions de la piste **pt**.

Exemple : si l'objet **p3** correspond à la piste 3 de l'exemple ci-dessus et contient la liste de régions **[r1, r2]** où **r1** correspond à la région « Intro » et **r2** à la région « Refrain », alors **p3.add_region("Deuxième intro", f, 4, 10, 12)** (où **f** est un objet **Fichier** donné) créera une nouvelle région **r3**, insérée entre **r1** et **r2** dans la liste de régions, à la position 16 (car 4 est inférieur à la position de fin de **r1**) et provoquera le décalage de la région **r2** à la position $16 + (12 - 10) = 18$.

4. Définir la classe **Piste** en respectant le cahier des charges ci-dessus.